

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут ім. Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЛАБОРАТОРНА РОБОТА №1
З ДИСЦИПЛІНИ “Системне програмування”

Виконав:
студент групи ІО-33
Герасимчук Д. С.
Перевірив:
Гордієнко Н. Ю.

м. Київ – 2026 р.

1. Мета лабораторної

Головним завданням цієї роботи було підготувати основу, на якій далі будуватиметься весь проєкт. Треба було вибрати плату й середовище, створити проєкт і зробити так, щоб прошивку можна було зібрати й запустити без ручної рутини: збірка в Docker, запуск у симуляторі і перегляд логів по UART - кожне однією командою.

Після цього потрібно було переконатися, що все справді працює: мікроконтролер відповідає по UART, а інтерфейси, на яких триматимуться датчики проєкту, піднімаються й реагують на зміни в схемі. В процесі було обрано тему проєкту й описати її в PRD, щоб наступні лабораторні мали конкретну ціль, а всю роботу вести в git.

2. Процес виконання

2.1. Журнал робіт

Дата	Виконані задачі
19.09	Розбір завдання на лабораторну роботу. Зібрано Docker-образ з тулчейном і Makefile-обгортку, намальовано схему у Wokwi. Під час перших запусків прошивки в симуляторі отримано вивід по UART і знайдено особливості моделі Wokwi для ADC та I2C.
23.09	Конфігурацію в CubeMX доведено до фінального вигляду (тактування від HSI, усі піни, режим ADC, переривання кнопки) і згенеровано фінальний код. Логіку прошивки додано в блоки USER CODE, виправлено обробку кнопки.
25.09	Скріншоти, документація (hardware-components.md та PRD.md), оформлення git-репозиторію.

2.2. Збірка і запуск

Тулчейн це образ на ubuntu:22.04 з gcc-arm-none-eabi, libnewlib-arm-none-eabi, srecord. Кореневий Makefile запускає firmware/Makefile (згенерований CubeMX) у контейнері з --user \$(id -u):\$(id -g), щоб файли в build/ належали користувачу, а не root.

- make image - один раз зібрати образ;
- make build - скомпілювати прошивку в firmware/build/firmware.elf;

Рис. 1. Pinout у CubeMX: усі використані піни і їхні функції.

Тактування: внутрішній HSI48, дільник /1, SYSCLK = HCLK = PCLK = 48 МГц, Flash latency 1 WS.

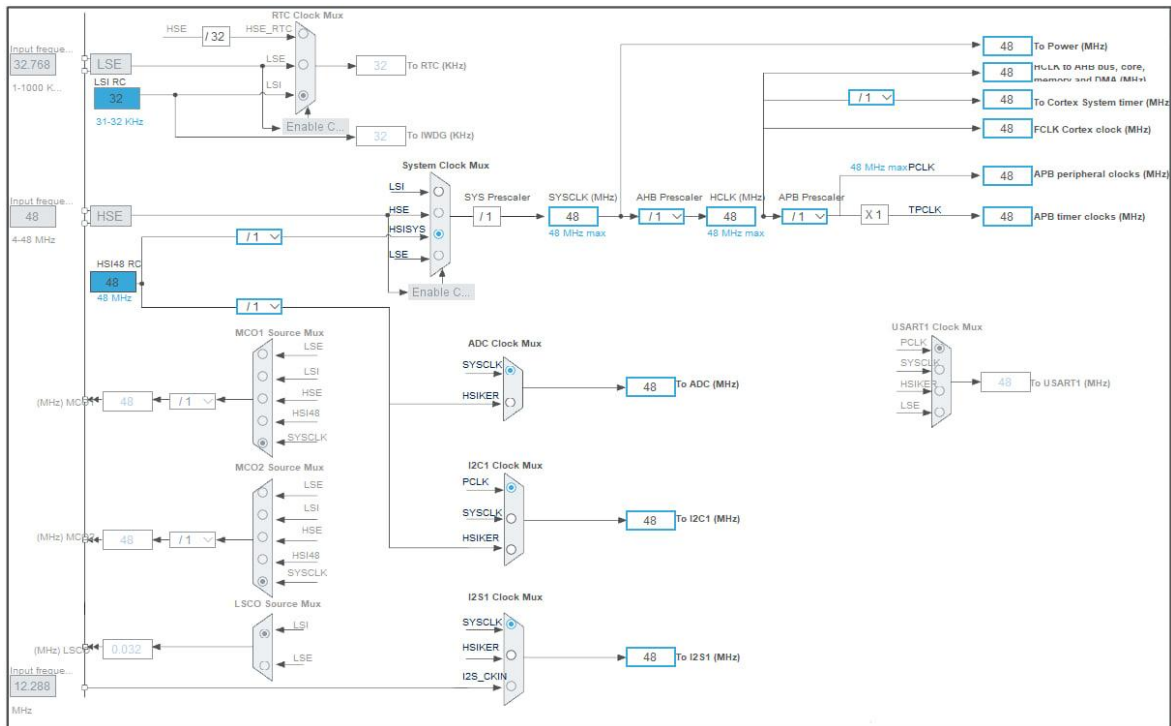


Рис. 2. Clock Configuration: System Clock Mux на HSISYS, SYSCLK 48 МГц.

2.4. Перша відповідь по UART і перевірка інтерфейсів

Власний код лежить у блоках USER CODE файлу main.c, тож регенерація в CubeMX його не затирає. Вивід printf через _write() іде в USART2. Після старту прошивка друкує частоту і стан кожної периферії, потім hello from STM32, а далі раз на секунду tick N зі значеннями обох каналів АЦП і перемикає LD4. При старті й кожні 5 секунд вона сканує шину I2C і робить холосту SPI-транзакцію, а по натисканню кнопки друкує button pressed.

```
NUCLEO-C031C6, SYSCLK 48000000 Hz
USART2 HAL_OK
I2C1 HAL_OK
SPI1 HAL_OK
ADC1 HAL_OK
TIM3 HAL_OK
hello from STM32
i2c scan: 0x68 0x69
spi ping: ok
tick 1 adc0=1001 adc1=2148 led=on
tick 2 adc0=1001 adc1=2172 led=off
tick 3 adc0=1001 adc1=2342 led=on
tick 4 adc0=1001 adc1=2308 led=off
tick 5 adc0=3912 adc1=2104 led=on
i2c scan: 0x68 0x69
spi ping: ok
tick 6 adc0=3912 adc1=2108 led=off
tick 7 adc0=3605 adc1=2264 led=on
tick 8 adc0=89 adc1=2330 led=off
tick 9 adc0=89 adc1=2338 led=on
tick 10 adc0=89 adc1=2471 led=off
i2c scan: 0x68 0x69
spi ping: ok
button pressed
button pressed
tick 11 adc0=89 adc1=2471 led=on
□
```

Рис. 3. Вивід UART у VS Code: HAL_OK на кожну периферію, tick N щосекунди, реакція на кнопку.

Результати перевірок:

- I2C: скан знаходить MPU-6050 (0x68; модель Wokwi відповідає ще й на 0x69).
- ADC: при 500 лк фоторезистор дає 1001. Розрахунок для дільника дає ~1002. При зміні освітленості значення змінюється у правильний бік: темніше - більше. PA1 не підключений, тому adc1 плаває.
- Кнопка: при натисканні прінтється повідомлення button pressed.

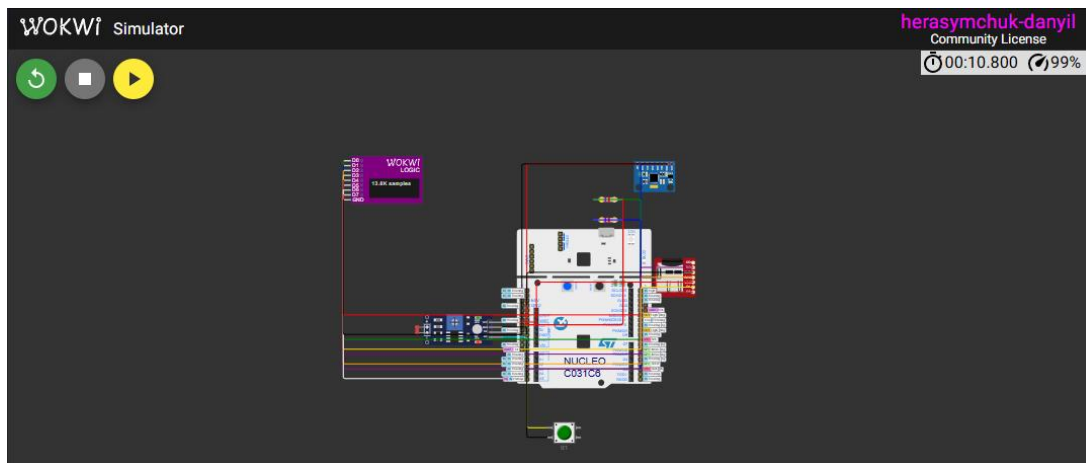


Рис. 4. Схема в симуляторі.

Логічний аналізатор підключений до SCL, SDA, SCK, MOSI, MISO, CS, TX UART і LD4. Після зупинки симуляції Wokwi записує сигнали у docs/wokwi.vcd.

```
incident-recorder/docs/wokwi.vcd
1  $version Generated by Wokwi.com $end
2  $date Fri, 25 Sep 2026 16:24:03 GMT $end
3  $timescale 1ns $end
4  $scope module logic $end
5  $var wire 1 ! D0 $end
6  $var wire 1 ' D1 $end
7  $var wire 1 # D2 $end
8  $var wire 1 $ D3 $end
9  $var wire 1 % D4 $end
10 $var wire 1 & D5 $end
11 $var wire 1 ' D6 $end
12 $var wire 1 ( D7 $end
13 $upscope $end
14 $enddefinitions $end
15 #0
16 1!
17 #0
18 1"
19 #0
20 0#
21 #0
22 0$
```

Рис. 5. Початок VCD-файлу з логічного аналізатора.

2.5. Коміти

Робота велась у гілці [lab1](#):

- [ab438c9](#) - Docker-тулчейн і Makefile для збірки;
- [9cd0c72](#) - схема і конфіг Wokwi, цілі sim/monitor/flash;
- [9ad05e7](#) - проєкт, згенерований у CubeMX;
- [c536bcf](#) - вивід по UART, перевірка I2C/SPI, АЦП, кнопка;
- [5225fa6](#) - скріншоти;
- [1952c20](#) - опис апаратних компонентів;
- [207a82d](#) - PRD.

2.6. Проблеми та їх вирішення

Проблема 1: у Wokwi немає NUCLEO-F411RE/F446RE

Wokwi підтримує лише три STM32: C031C6, L031K6 і F103C8, тому було обрано NUCLEO-C031C6 - повноцінну Nucleo-64 з пакетом у CubeMX. Процес розробки лишився тим самим, змінились лише HAL і лінкер-скрипт, а ядро тепер Cortex-M0+ без FPU з 32 КБ Flash.

Проблема 2: схема Wokwi не проходила перевірку

wokwi-cli не приймав пін PB0: на платі він виведений двічі, тому в моделі Wokwi називається PB0.1 і PB0.2. У diagram.json використано nucleo:PB0.1.

Проблема 3: АЦП повертав значення не того каналу

Модель АЦП у Wokwi не вміє перетворювати кілька каналів за один запуск у режимі fixed sequencer і віддає значення останнього налаштованого. У CubeMX sequencer переведено в режим «fully configurable», а функція adc_read() вибирає потрібний канал перед кожною конверсією.

Проблема 4: запис по I2C через HAL завжди падав у таймаут

HAL записує перший байт у регістр TXDR ще до генерації START, модель Wokwi на цьому зупиняється, і шина лишається зайнятою. Запис зроблено напряму через регістри CR2, ISR і TXDR за описом «Master transmitter» з RM0490, а ініціалізація I2C лишилася згенерованою CubeMX.

Проблема 5: скан I2C «знаходив» усі 112 адрес

У моделі Wokwi транзакція без даних не доходить до фази адреси, тому NACK не приходить і «відповідають» усі адреси. Пробу зроблено одним байтом даних, і тепер скан показує тільки адреси MPU-6050 (0x68 0x69).

3. Висновок

У результаті роботи є готова основа для проєкту: прошивка збирається в Docker однією командою, запускається в симуляторі Wokwi і виводить логи по UART, а всі потрібні інтерфейси (I2C, SPI, ADC, GPIO) налаштовані й перевірені на реальних компонентах схеми.

Під час роботи я розібрався, як CubeMX генерує код ініціалізації і як додавати в нього власну логіку так, щоб вона не зникла після регенерації. Також зрозумів, як налаштовується тактування мікроконтролера і навіщо при підвищенні частоти потрібна затримка доступу до Flash, та навчився перевіряти призначення пінів за таблицями з datasheet.

Найважливіший практичний висновок це те, що симулятор поводить ся не зовсім так, як справжня плата. Проблеми з АЦП, записом по I2C і кнопкою були спричинені саме особливостями моделі Wokwi, а не помилками в коді. Щоб їх знайти, довелося дивитися на стан регістрів периферії, а не лише на те, що повертають функції HAL.

4. Матеріали ти посилання

1. Методичка до LP1, курс «Системне програмування вбудованих систем».
2. [STMicroelectronics. DS13867 Rev 3 - STM32C031x4/x6 datasheet.](#)
3. [Wokwi Docs. STM32 Nucleo64 C031C6.](#)
4. [Wokwi Docs. Configuring Your Project \(wokwi.toml\).](#)
5. [Wokwi Docs. Wokwi CLI Usage.](#)
6. [Wokwi Docs. Logic Analyzer.](#)
7. [Wokwi. Приклад «Nucleo-C031C6 and I2C LCD1602».](#)

5. Відеозвіт:

https://drive.google.com/file/d/1OpWqXC5uCBXOHKpq7Z7wm8U9hyCmyG_e/view?usp=drive_link

6. RPD

PRD винесено в окремий файл репозиторію: [docs/PRD.md](#).